

Leaflet 學習入門

講師：林新德 shinder.lin@gmail.com

講義版本：2026-09-10

範例專案：<https://github.com/shinder/html5-js-api-examples>

目錄

0. OpenStreetMap 生態系	p. 3
為什麼選 Leaflet (vs Google Maps)	p. 3
L010：基本地圖設定	p. 3
引入 Leaflet 函式庫	p. 3
地圖容器	p. 4
初始化 + 圖磚 + 標記	p. 4
L020：地圖標示	p. 4
自訂圖示	p. 4
比例尺	p. 5
圓形 (向量圖形)	p. 5
L030：取得目前位置	p. 5
Geolocation API 速覽	p. 5
與地圖搭配	p. 6
L040：持續追蹤位置	p. 7
L050：切換地圖圖層	p. 7
多個 tile layer	p. 7
Layers control	p. 8
L060：地名搜尋 (Geocoding)	p. 9
Nominatim API	p. 9
找不到的處理	p. 9
L070：路線規劃 (Routing)	p. 9
OSRM API	p. 10
用 L.geoJSON 畫路線	p. 10
L080：反向地址查詢 (Reverse Geocoding)	p. 10
監聽地圖點擊	p. 11
Nominatim reverse 端點	p. 11
獨立 popup	p. 11
L090：區域搜尋 (Overpass API)	p. 11
關於 Overpass	p. 11
Overpass QL (query language)	p. 11

發送請求	p. 12
處理結果	p. 12
L100：從本機 JSON 載入多個標記	p. 12
為什麼把資料離線化	p. 13
L110：連動選單與地圖	p. 13
建立 UI ↔ 資料的對應關係	p. 13
事件委派	p. 14
經緯度之間求距離	p. 14

0. OpenStreetMap 生態系

「在網頁上放地圖」這件事其實由幾個獨立但常被搭配使用的開源工具組成：

角色	名字	做什麼
圖資	OpenStreetMap (OSM)	由社群共同維護的全球地理資料庫
圖磚 / 渲染	OpenStreetMap Tile Server 等	把 OSM 資料渲染成 256×256 PNG 圖磚
前端函式庫	Leaflet	JavaScript 地圖元件，把圖磚拼起來、處理互動
地名 ↔ 座標	Nominatim	文字地名 → 經緯度 (geocoding)
路線規劃	OSRM	兩點之間最短路徑
圖資查詢	Overpass API	用 query language 查 OSM 圖資 (咖啡店、加油站...)

這些服務都有「公共免費版」（適合 demo），但**都有使用政策與速率上限**；正式環境量大時建議自架或改用商業服務（Mapbox、HERE、Google Maps）。

為什麼選 Leaflet (vs Google Maps)

項目	Leaflet + OSM	Google Maps
費用	免費	有免費額度，超過收費
API key	不需要	需要、要綁信用卡
客製圖磚	可換任何 tile 來源	受限於 Google 提供樣式
圖資完整度	視地區，台灣 OK	全球品質一致
Bundle size	~150 KB	由 Google 載入

教學、原型、不想處理 API 計費時 Leaflet 較合適。

L010：基本地圖設定

範例 `public/L010-basic.html` 展示初始化一個地圖、加圖磚、加標記三個基本步驟。

引入 Leaflet 函式庫

CSS 放 `<head>`、JS 放 `<body>` 結尾（或用 `defer`）：

```
<link rel="stylesheet" href="https://unpkg.com/leaflet@1.9.4/dist/leaflet.css" />
<script src="https://unpkg.com/leaflet@1.9.4/dist/leaflet.js"></script>
```

Leaflet 的 CSS **不可省略** — 沒它的話 marker 會跑位、tile 拼接會錯亂。

版本要釘死：網址寫 `leaflet@1.9.4`，不要只寫 `leaflet`（那會拿到「最新版」）。
Leaflet 2.0 已進入 alpha，改成 ESM 模組、全域變數 `L` 不再是核心的一部分、
`L.marker()` 要改寫成 `new Marker()`。一旦 2.0 成為預設版本，沒釘版本的網頁會整個壞掉。本講義所有範例都以 1.9.4 為準。

地圖容器

需要一個有 ID 與明確高度的 `<div>`。Leaflet 不會自己判斷高度，沒設高度地圖看不到。

```
<div id="map"></div>
<style>
  #map { height: 500px; width: 100%; }
</style>
```

初始化 + 圖磚 + 標記

```
const taipei101 = [25.033, 121.5654]; // Leaflet 座標格式：[緯度, 經度]
const map = L.map("map").setView(taipei101, 13);

L.tileLayer("https://tile.openstreetmap.org/{z}/{x}/{y}.png", {
  maxZoom: 19,
  attribution: '© <a href="https://www.openstreetmap.org/copyright">OpenStreetMap</a>',
}).addTo(map);

L.marker(taipei101).addTo(map);
```

關鍵概念：

- **座標格式**：Leaflet 用 `[緯度, 經度]`（latitude first）。但 OSRM、GeoJSON 用 `[經度, 緯度]`（longitude first）—— 兩個常搞混。
- **Zoom level**：數字越大畫面越近。0 是整顆地球、19 是街區門牌等級、20+ 部分 tile 服務不支援。
- **Tile URL 變數**：`{z}/{x}/{y}` 是 zoom 與 tile 座標，由 Leaflet 自動填入。有些圖磚服務網址還有 `{s}`（subdomain，a/b/c 輪用，Leaflet 也會自動代入）；OSM 官方已改用單一網域 `tile.openstreetmap.org`，不需要 `{s}`。
- **attribution**：依 [OSM 使用政策](#) 是必填項目（顯示版權）。政策同時要求網頁要送出正常的 `Referer`（瀏覽器預設會送，別把頁面設成 `no-referrer`），並禁止大量預先下載圖磚。

L020：地圖標示

範例 `public/L020-labels.html` 加上自訂 marker 圖示、popup、比例尺、圓形區域。

自訂圖示

預設 marker 是藍色水滴；用 `L.icon` 換成自己的圖：

```
const customIcon = L.icon({
  iconUrl: "./images/mario.gif",
  iconSize: [40, 40], // 圖示寬高
  iconAnchor: [20, 40], // 「錨點」對應到地理座標的點。底部中點 = 站立感
  popupAnchor: [0, -34], // popup 相對於 anchor 的位置 (往上 34px)
});

L.marker(taipei101, { icon: customIcon })
  .addTo(map)
  .bindPopup("<b>台北 101</b><br>台北市信義區")
  .openPopup(); // 預設就開
```

iconAnchor 不調好：marker 圖會浮在實際座標旁，移動地圖時感覺整顆飄。一般地標型圖示放 [寬/2, 高] (底部中點)；圓點型放 [寬/2, 高/2] (中心)。

比例尺

```
L.control
  .scale({ metric: true, imperial: false, maxWidth: 200 })
  .addTo(map);
```

公制為主、隱藏英制；maxWidth 是視覺寬度上限。

圓形（向量圖形）

Leaflet 除了 marker，也能畫 SVG 向量：

```
L.circle([25.033, 121.5654], {
  color: "orange",
  fillColor: "yellow",
  fillOpacity: 0.5,
  radius: 500, // 公尺
}).addTo(map);
```

其他向量：L.polyline、L.polygon、L.rectangle、L.geoJSON (看 L070)。

L030：取得目前位置

範例 public/L030-current-position.html 用 navigator.geolocation 取使用者座標，移到地圖中心。

Geolocation API 速覽

navigator.geolocation 讓網頁取得使用者目前的位置。這是隱私權資料，瀏覽器會跳出權限對話框由使用者授權。精準度依來源而定：有 GPS 的裝置最準（誤差幾公尺），其次是 WiFi 定位（幾十公尺），最差只能用 IP 反查（幾公里）。

- getCurrentPosition(success, error, options)：取一次位置。

- `watchPosition(success, error, options)`：位置變動時持續觸發，回傳 `watchId`，用 `clearWatch(watchId)` 停止（見 L040）。
- `position.coords`：除了 `latitude`、`longitude`，還有 `accuracy`（公尺）、`altitude`、`heading`、`speed`；裝置不支援的欄位為 `null`。
- 錯誤物件 `error.code`：1 = `PERMISSION_DENIED`、2 = `POSITION_UNAVAILABLE`、3 = `TIMEOUT`。

`options` 物件：

屬性	說明
<code>enableHighAccuracy</code>	要求高精度（會吃 GPS、較耗電）
<code>timeout</code>	等待毫秒數，超時觸發 <code>error</code> 且 <code>code === 3</code>
<code>maximumAge</code>	接受多舊的快取結果（毫秒），0 表示強制重新取

HTTPS 限制：Geolocation API 只有在 `localhost` 或 HTTPS 才會回座標。其他情境會直接觸發 `error callback`、`code === 1`（`PERMISSION_DENIED`）。

要在手機上測真正的 GPS，需要一個外部可達的 HTTPS 網址。用 [Cloudflare Tunnel](#) 的 `quick tunnel` 最省事：免帳號、免費、一行指令，把本機 `server` 映射成公開 HTTPS 網址。

```
brew install cloudflared # macOS; Windows: winget install
--id Cloudflare.cloudflared
cloudflared tunnel --url http://localhost:3031 # 範例專案的 Express 跑在 3031
```

幾秒後會印出 `https://<隨機名>.trycloudflare.com`，手機開這個網址即可。每次重啟網址都會變，也沒有服務保證，只適合測試用。

與地圖搭配

```
const options = {
  enableHighAccuracy: true, // 用 GPS (耗電但準)
  timeout: 5000,           // 等待毫秒
  maximumAge: 0,          // 不用快取，要最新的
};

navigator.geolocation.getCurrentPosition(
  (position) => {
    const lat = position.coords.latitude;
    const lon = position.coords.longitude;
    map.setView([lat, lon], 16);
    L.marker([lat, lon]).addTo(map).bindPopup("您的當前位置").openPopup();
  },
  (error) => {
    // code: 1=PERMISSION_DENIED 2=POSITION_UNAVAILABLE 3=TIMEOUT
  }
);
```

```
    console.error("GPS 錯誤:", error.code, error.message);
  },
  options
);
```

L040：持續追蹤位置

範例 `public/L040-watch-position.html` 把 `getCurrentPosition` 改成 `watchPosition`，使用者移動時持續更新。

```
let marker; // 只建一個 marker，之後只更新位置

const watchId = navigator.geolocation.watchPosition(
  (position) => {
    const latlng = [position.coords.latitude, position.coords.longitude];
    map.setView(latlng, 16);

    // 第一次建立 marker，之後只用 setLatLng 更新位置
    if (!marker) {
      marker = L.marker(latlng).addTo(map).bindPopup("您的目前位置").openPopup();
    } else {
      marker.setLatLng(latlng);
    }
  },
  (error) => console.error("GPS 錯誤:", error),
  options
);

// 需要停止追蹤時：
// navigator.geolocation.clearWatch(watchId);
```

關鍵差別：

- `getCurrentPosition` 觸發一次；`watchPosition` 在位置變動時持續觸發。
- `watchPosition()` 回傳一個 `watchId`，需要時可以 `navigator.geolocation.clearWatch(watchId)` 停止。
- Marker 只建一次，之後用 `marker.setLatLng()` 搬位置。若每次回呼都 `L.marker().addTo(map)`，地圖會隨著移動堆滿 marker；「先 `map.removeLayer()` 移除舊的再建新的」也能用，但多做了一次 DOM 增刪，沒有必要。

L050：切換地圖圖層

範例 `public/L050-tile-layers.html` 提供「標準地圖 / 地形圖」切換 UI。

多個 tile layer

每種樣式建一個 `L.tileLayer` 物件，先不 `.addTo(map)`：

```

const osmStandard = L.tileLayer(
  "https://tile.openstreetmap.org/{z}/{x}/{y}.png",
  { attribution: "© OpenStreetMap contributors" }
);
const osmTopo = L.tileLayer(
  "https://{s}.tile.opentopomap.org/{z}/{x}/{y}.png",
  { attribution: "© OpenTopoMap, © OpenStreetMap contributors" }
);
// 臺灣通用電子地圖：內政部國土測繪中心「國土測繪圖資服務雲」WMTS
const nlscEmap = L.tileLayer(
  "https://wmts.nlsc.gov.tw/wmts/EMAP/default/GoogleMapsCompatible/{z}/{y}/{x}",
  {
    maxZoom: 18,
    attribution: '底圖 <a href="https://maps.nlsc.gov.tw/">國土測繪圖資服務雲</a>',
  }
);

```

臺灣通用電子地圖（NLSC）：內政部國土測繪中心提供，免申請、免費、中文地名最齊全，只要標出處即可使用。跟 OSM 不同的地方：

- 網址是 `{z}/{y}/{x}`，**y 在 x 前面**（WMTS 的 TileRow / TileCol 慣例）。寫反不會報錯，只會拿到一片空白或錯的地區。
- 沒有 `{s}`，也沒有副檔名。
- `maxZoom` 是 18。
- EMAP 是標準版；把網址中的 EMAP 換成 EMAP16（無等高線及門牌，畫面較乾淨）、EMAP6（無等高線）、PHOT02（正射影像，即空拍圖）就能切換圖層。完整清單在 WMTS Capabilities：<https://wmts.nlsc.gov.tw/wmts?SERVICE=WMTS&REQUEST=GetCapabilities>

Layers control

把 layer 對應成「顯示名稱 → 物件」物件，丟給 `L.control.layers`：

```

const baseMaps = {
  "標準地圖": osmStandard,
  "地形圖": osmTopo,
  "臺灣通用電子地圖": nlscEmap,
};

osmStandard.addTo(map); // 預設顯示
L.control.layers(baseMaps).addTo(map); // 加切換 UI

```

`L.control.layers(baseLayers, overlays)`：

- 第一個參數 `baseLayers`：**互斥選擇**（radio），一次只顯示一個 base map。

- 第二個參數 `overlays`：可疊加選擇 (checkbox)，例如「顯示便利商店」「顯示腳踏車道」。

更多免費 tile 來源：[Leaflet provider preview](#)，包含 CARTO Light/Dark、Esri 衛星圖等。以前常見的 Stamen 藝術風圖磚自 2023 年起改由 Stadia Maps 提供，要註冊取得 API key (有免費額度) 才能用。

L060：地名搜尋 (Geocoding)

範例 `public/L060-name-search.html` 把「台北車站」這種文字地址轉成經緯度，叫做 **geocoding**。這個範例本身沒用 Leaflet，純粹示範 Nominatim API；取到座標後想標到地圖上自己接 L010 即可。

Nominatim API

```
const url = `https://nominatim.openstreetmap.org/search?format=json&q=${encodeURIComponent(address)}`;  
const response = await fetch(url);  
const data = await response.json(); // 陣列，可能為空
```

回傳是陣列，`data[0]` 通常是最佳結果，包含 `lat`、`lon` (字串，要 `parseFloat`)、`display_name` 等欄位。

使用政策：Nominatim 公共服務限制每秒 1 次請求，並要求能識別呼叫者。從後端呼叫時要自訂 `User-Agent` 標頭寫上你的應用名稱；從瀏覽器呼叫時 `fetch` 無法修改 `User-Agent` (屬於瀏覽器禁止改寫的標頭)，改由瀏覽器自動送出的 `Referer` 識別，不必另外處理。商業使用建議自架或改用 [LocationIQ](#)、[Mapbox Geocoding](#) 等付費服務。政策文件：<https://operations.osmfoundation.org/policies/nominatim/>

找不到的處理

```
const result = await geocode(name);  
if (!result) {  
  alert(`找不到「${name}」`);  
  return;  
}
```

實務上要顯示在 UI 上、不要用 `alert` 阻塞使用者。

L070：路線規劃 (Routing)

範例 `public/L070-route-planning.html` 用 OSRM 計算兩點之間的駕車路徑，把路線畫到地圖上。

OSRM API

URL 形式特別：

```
https://router.project-osrm.org/route/v1/{profile}/{lon1},{lat1};{lon2},{lat2}?
overview=full&geometries=geojson
```

- `{profile}`：`driving`（汽車） / `walking`（步行） / `cycling`（自行車）。公共 OSRM 預設只有 `driving`。
- 座標：經度, 緯度 順序，跟 Leaflet 反過來。第一個常見錯。
- `overview=full`：要完整路徑點（不要只回起點/終點）。
- `geometries=geojson`：要 GeoJSON 格式（Leaflet 可以直接畫）。

```
const url = `https://router.project-osrm.org/route/v1/driving/`
  + `${start[1]},${start[0]};${end[1]},${end[0]}`
  + `?overview=full&geometries=geojson`;
const response = await fetch(url);
const data = await response.json();
const route = data.routes[0]; // 第一條建議路線
```

`route` 重要欄位：

- `route.geometry`：GeoJSON LineString，可以直接餵給 `L.geoJSON()`
- `route.distance`：總長度（公尺）
- `route.duration`：預估時間（秒）

用 L.geoJSON 畫路線

```
L.geoJSON(route.geometry, {
  style: { color: "#0080ff", weight: 5, opacity: 0.8 },
}).addTo(map);

const distance = (route.distance / 1000).toFixed(1);
const duration = Math.round(route.duration / 60);
info.innerText = `距離: ${distance} km, 時間: ${duration} 分鐘`;
```

公共 OSRM 服務也有用量限制，正式服務應自架。商業替代：[Mapbox Directions](#)、[GraphHopper](#)、[OpenRouteService](#)。

L080：反向地址查詢（Reverse Geocoding）

範例 `public/L080-reverse.html` 反過來：點地圖任一點 → 查地址。常用於「點圖選地點」UX。

監聽地圖點擊

```
map.on("click", async (e) => {
  const { lat, lng } = e.latlng; // 注意: Leaflet 用 lng (不是 lon)
  const address = await reverseGeocode(lat, lng);
  // ...
});
```

Nominatim reverse 端點

```
const url = `https://nominatim.openstreetmap.org/reverse`
  + `?format=json&lat=${lat}&lon=${lon}&zoom=18&addressdetails=1`;
```

- `zoom` : 要求多細的位置 — 18 = 門牌等級。
- `addressdetails=1` : 回傳結構化的地址欄位 (`address.country` / `address.city` / ...) 。

獨立 popup

之前的 popup 都綁在 marker 上；也可以建獨立 popup 直接開在地圖任一點：

```
L.popup()
  .setLatLng(e.latlng)
  .setContent(`<b>地址:</b>${address.display_name}`)
  .openOn(map);
```

`openOn(map)` 會自動關掉地圖上其他 popup，適合「同時只開一個」的情境。

L090：區域搜尋 (Overpass API)

範例 `public/L090-search-cafe.html` 用 Overpass API 在指定 bounding box 內搜尋符合條件的 OSM 圖資（這裡是「咖啡店」）。

關於 Overpass

- OSM 主體是 OpenStreetMap Foundation (OSMF)，非營利組織。
- `overpass-api.de` 不是 OSMF 官方運營，而是社群自架服務、免費對外提供。沒有 SLA，正式應用建議自架 ([overpass-api/Overpass-API on GitHub](https://overpass-api.de/about)) 。
- 文件：https://wiki.openstreetmap.org/wiki/Overpass_API/Overpass_QL

Overpass QL (query language)

```
[out:json][timeout:25];
(
  node["amenity"="cafe"](25.0,121.5,25.1,121.6);
  way["amenity"="cafe"](25.0,121.5,25.1,121.6);
  relation["amenity"="cafe"](25.0,121.5,25.1,121.6);
);
out center;
```

逐行解釋：

- `[out:json]`：要 JSON 輸出（也可以是 XML）
- `[timeout:25]`：25 秒超時
- 三個類型 `node`、`way`、`relation` 是 OSM 圖資的三種基本元素：
 - **node**：一個點（咖啡店、紅綠燈、銀行）
 - **way**：一條路線或封閉多邊形（路、河、建物外框）
 - **relation**：多個 node/way 的組合（公車路線、行政區）
- `["amenity"="cafe"]`：tag filter，篩出 `amenity=cafe`
- `(25.0,121.5,25.1,121.6)`：bounding box，格式（南緯,西經,北緯,東經）
- `out center;`：輸出。對 way/relation 計算一個中心點放在 `center` 屬性，方便用點標示

發送請求

Overpass 用 POST 接收查詢，body 的 key 是 `data`：

```
const response = await fetch("https://overpass-api.de/api/interpreter", {
  method: "POST",
  body: "data=" + encodeURIComponent(query),
});
```

處理結果

```
data.elements.forEach((cafe) => {
  let lat, lon;
  if (cafe.type === "node") {
    lat = cafe.lat;
    lon = cafe.lon;
  } else if (cafe.center) { // way / relation
    lat = cafe.center.lat;
    lon = cafe.center.lon;
  }
  if (lat && lon) {
    const name = cafe.tags?.name || "咖啡店";
    L.marker([lat, lon]).addTo(map).bindPopup(`☺️ ${name}`);
  }
});
```

常用的 OSM tag：`amenity=restaurant` / `=hospital` / `=school` / `=atm`，
`shop=convenience` / `=supermarket`，`tourism=hotel` / `=museum`。完整清單：https://wiki.openstreetmap.org/wiki/Map_features。

L100：從本機 JSON 載入多個標記

範例 `public/L100-markers.html` 與 L090 結果視覺一致，但資料源從即時 API 改成預先存好的 `data/cafes.json`。

為什麼把資料離線化

考量	即時 API (L090)	本機 JSON (L100)
速度	慢 (要打外部)	快 (同源檔)
穩定	看 API server	跟你的 server 同生死
流量	每次都打	只打一次
即時性	最新	凍結在抓取那一刻

實務工作流：開發初期先打 Overpass，一次抓出來存成 JSON 檔，網頁直接讀檔。資料變動頻繁才需要每次都打 API。

```
fetch(`data/cafes.json`)
  .then((r) => r.json())
  .then((result) => {
    result.elements.forEach((cafe) => {
      if (cafe.type !== "node") return;
      const name = cafe.tags?.name || "咖啡店";
      L.marker([cafe.lat, cafe.lon]).addTo(map).bindPopup(`📍 ${name}`);
    });
  });
```

注意 JSON 格式刻意跟 Overpass 回傳一樣（elements 陣列），讓「換資料源」幾乎零改動。這就是把資料獲取與資料呈現分離的好處。

L110：連動選單與地圖

範例 `public/L110-menu-to-loc.html` 結合「側邊清單」與「地圖標記」：點清單項目 → 地圖飛到對應位置 + 開 popup。

建立 UI ↔ 資料的對應關係

關鍵在於記住「哪個 DOM 元素」對應「哪個 marker」。用 `Map`（key 可以是任何型別，包括 DOM 元素物件）最直接：

```
const menuMap = new Map(); // DOM 元素 → marker / 座標

const cafes = elements
  .filter((c) => c.type === "node")
  .sort(() => Math.random() - 0.5)
  .slice(0, 20); // 隨機取 20 家

for (const cafe of cafes) {
  const name = cafe.tags?.name || "咖啡店";

  const marker = L.marker([cafe.lat, cafe.lon])
    .addTo(map)
    .bindPopup(`📍 ${name}`);
```

```
const item = document.createElement("div");
item.className = "item";
item.innerText = name;
list.append(item);

menuMap.set(item, { lat: cafe.lat, lon: cafe.lon, marker });
}
```

第一行用 `.filter().sort().slice()` 三個操作鏈起來，讀起來就是「過濾出 node、隨機排、取前 20」，比 `for + counter + break` 直覺。

事件委派

不要一個一個 item 掛 listener — 在父容器掛一個 listener，從 `e.target` 判斷實際被點到的元素：

```
list.addEventListener("click", (e) => {
  const data = menuMap.get(e.target);
  if (!data) return; // 點到 list 本身 (item 之間的空隙) 就忽略

  data.marker.openPopup();
  map.setView([data.lat, data.lon], 16);
});
```

事件委派的好處：item 動態增刪都不用重綁、記憶體佔用少、程式碼簡潔。

經緯度之間求距離

地球是橢球體，不能用平面幾何公式算兩點距離。Haversine formula 把地球當成正球體，誤差在 $< 1\%$ （幾公里等級），對一般應用足夠。

```
function haversine(lat1, lon1, lat2, lon2) {
  const R = 6371; // 地球半徑 (km)
  const toRad = (deg) => (deg * Math.PI) / 180;

  const dLat = toRad(lat2 - lat1);
  const dLon = toRad(lon2 - lon1);

  const a =
    Math.sin(dLat / 2) ** 2 +
    Math.cos(toRad(lat1)) * Math.cos(toRad(lat2)) * Math.sin(dLon / 2) ** 2;

  return R * 2 * Math.asin(Math.sqrt(a)); // 公里
}

// 台北 (25.033, 121.565) → 高雄 (22.627, 120.301)
```

```
console.log(haversine(25.033, 121.565, 22.627, 120.301).toFixed(2) + " km");  
// 約 295 km
```

要更高精度（例如測量、導航）用橢球模型 — Vincenty formula 或基於 WGS84 的 geodesic 計算。

Leaflet 內建：`map.distance([lat1, lon1], [lat2, lon2])` 也算 haversine，回傳公尺。